

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 1 / 11

REVISION HISTORY

Date	Version	Description	Author
20/02/26	1.0	Identification and Software Development Activities (intro) parts are filled	Hamza
21/02/26	1.1	Responsibilities part filled	Genidy
22/02/26	1.2	Risk management	Emir, Bora, Hamza, Genidy
22/02/26	1.3	Software Development Activities (intro) part completed, Gantt Chart added and Risk Management Table revised	Emir, Bora
01/03/26	1.4	Report revised through the feedback	Bora, Genidy
01/04/26	1.5	Gantt Chart revised	Emir, Hamza

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 2 / 11

TABLE OF CONTENTS

1 Identification	3
1.1 Document overview	3
1.2 Abbreviations	3
1.2.1 Abbreviations	3
1.2.2 References	3
1.2.3 Project References	3
2 Software Development Activities	3
2.1 Software development process	3
2.1.1 Overview of process phases	4
2.1.2 Technical documentation	4
2.1.3 Deliverables	5
2.2 Software development tools	5
2.2.1 Workstation	5
2.2.2 Requirements management and documentation	5
2.2.3 Software Design	5
2.2.4 Coding and automated tests	5
2.2.5 Configuration management	6
2.3 Software development rules and standards	6
3 Responsibilities	6
3.1 Activities and responsibilities	6
4 Risk Assessment	7
4.1 Risk Analysis	7
4.2 Risk Planning	8

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 3 / 11

1 Identification

1.1 Document overview

This document contains the software development plan of software SCMS.

Student Club Management System (SCMS) is a desktop application developed using Java language. It is intended to automate administrative work for university student clubs. The application will deal with member registration, event organization, and financial management. It is expected to provide a common platform for both the presidents of the clubs (Administrators) and students (Members) to communicate effectively.

1.2 Abbreviations

1.2.1 Abbreviations

SCMS: Student Club Management System
UML: Unified Modeling Language
IDE: Integrated Development Environment
JDK: Java Development Kit
SRS: Software Requirement Specification
STP: Software Test Plan
SDD: Software Design Document
STR: Software Test Report
SAR: Software Analysis Report

1.2.2 References

1.2.3 Project References

#	Document Identifier	Document Title
[R1]	SCMS-SRS	Software Requirements Specifications of SCMS
[R2]	SCMS-STP	Software Test Plan of SCMS
[R3]	SCMS-SDD	Software Detailed Design of SCMS
[R4]	SCMS-STR	Software Test Report of SCMS
[R5]	SCMS-SAR	Software Analysis Report of SCMS

2 Software Development Activities

The section lists and describes the software development activities of **SCMS** software development project.

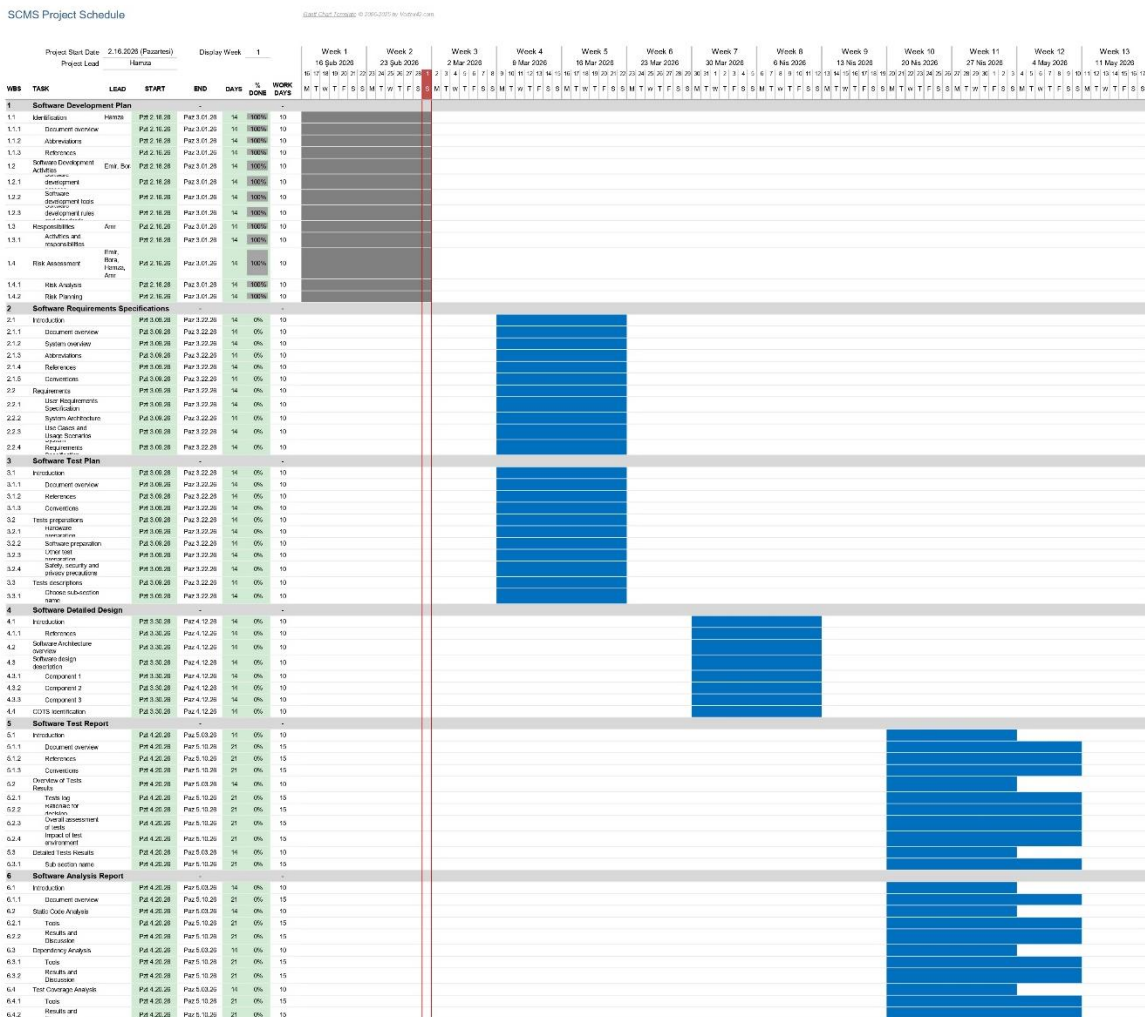
2.1 Software development process

This is a course project, which adopts the waterfall model as the software development process.

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 4 / 11

2.1.1 Overview of Process Phases

- **Planning:** Defining the SCMS scope, finalizing the SDP, and setting up GitHub.
- **Requirements Analysis:** Eliciting user requirements for club presidents and members to produce the SRS.
- **Design:** Designing the 3-package architecture and class diagrams for the **SDD**.
- **Implementation:** Coding using Java for the general architecture of the project, JavaFX for the GUI, and SQL for the database.
- **Testing and Analysis:** Unit testing (JUnit) and static/dependency analysis for **STR** and **SAR** steps.



(https://docs.google.com/spreadsheets/d/1D4UXYu7G1BpybXZCjtNA_SNTORv15t0p/edit?usp=drive_link&ouid=105807741878395193377&rtfpof=true&sd=true)

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 5 / 11

The following is a text representation of the project schedule showing the key milestones:

- February 23: Software Development Plan (SDP) first version
- March 2: SDP v2
- March 16: Software Requirements Document (SRS) and Software Test Plan (STP)
- March 23: SRS v2 and STP v2
- April 6: Software Design Document (SDD)
- April 13: SDD v2
- April to May: Implementation
- May 11: Software Test Report (STR), Software Analysis Report (SAR), and short demo

2.1.2 Technical documentation

The following documentation is produced during the software development phases to ensure traceability and quality:

- **Software Requirements Specification (SRS):** This document will specify the functional requirements (e.g., member registration, event organization) and non-functional requirements (e.g., system response time) of the SCMS.
- **Software Test Plan (STP):** This document will detail the testing plan, including hardware/software setup and the test scenarios (e.g., database connectivity test).
- **Software Detailed Design (SDD):** This document will detail the 3-package structure of the system, including UML class diagrams and sequence diagrams for main business processes such as budgeting.
- **Software Test Report (STR):** This document will note the result of the tests conducted, pointing out bugs or performance problems identified during the testing phase.
- **Software Analysis Report (SAR):** This document will give a summary of the result of the static code analysis, dependency analysis, and test coverage analysis to assess the overall quality of the code.

2.1.3 Deliverables

The following items will be delivered at the end of the process:

- Technical documentation as outlined in Section 2.1.2
- Software and its configuration files

2.2 Software development tools

2.2.1 Workstation

SCMS development will run on laptops or workstations with these minimum specs for smooth Java compilation, IntelliJ indexing, and local testing:

- **CPU:** Intel i5 or i7 (11th gen or higher) or AMD Ryzen 5 (5000 series or higher) with at least 4 cores
- **RAM:** 16 GB RAM

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 6 / 11

- **Storage:** 30 GB available space
- **OS:** Windows 10 (64-bit), macOS Ventura+, or Ubuntu 22.04 LTS

2.2.2 Requirements management and documentation

Google Docs (<https://docs.google.com>) and **Google Drive** (<https://drive.google.com>) will be used to write and manage the project and its requirements, and maintain an updated state of the process.

2.2.3 Software Design

The Unified Modeling Language (UML) will be the standard for all design documentation. To ensure this, the team will employ the following tools:

- **Argo UML:** An open-source UML modeling tool (<http://argouml.tigris.org/>).
- **Visual Paradigm:** A professional-grade UML design suite (<https://www.visual-paradigm.com/>).
- **Draw.io (diagrams.net):** A web-based diagramming software for flow mapping (<https://app.diagrams.net/>).
- **Microsoft Visio:** A specialized tool for architectural diagrams and flow mapping (<https://www.microsoft.com/en-us/microsoft-365/visio>).

2.2.4 Coding and automated tests

The SCMS is a Java-based desktop application that will require a strong technical foundation for its 3-package structure:

- **Java Development Kit (JDK):** This is the main development platform that will be used to create the application logic.
- **Integrated Development Environment (DE):** The group will leverage **IntelliJ IDEA** or **Eclipse** platforms to enable efficient coding and debugging.
- **GUI Framework:** Among JavaFX and Swing, **JavaFX** will be utilised as it is more modern, is constantly being maintained and has more features. The JavaFX framework will be used to create the Graphical User Interface, which will enable tasks such as member registration and event creation.
- **Automated Testing:** The main tool for unit tests is **JUnit**, which will ensure that all the project components are working properly and harmoniously before embarking on the integration stage.

2.2.5 Configuration management

GitHub¹ will be used for software configuration management and tracking issues regarding the software development. A public repository will be created for this purpose.

2.3 Software development rules and standards

The software design documentation will be done using UML. Moreover, the project will adhere to the following coding standards.

¹

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 7 / 11

2.3.1 Language Standards

Throughout the project, Official Oracle Java Code Conventions will be used. For a comprehensive list of these standards, refer to the official documentation located on Oracle's website: [Oracle Java Code Conventions](#).

- Naming conventions:
 - Classes: *PascalCase* (e.g. StudentClub)
 - Methods & Variables: *camelCase* (e.g. registerMember)
 - Constants: *UPPER_SNAKE_CASE* (e.g. ADMINISTRATION_BUDGET_LIMIT)
- Indentation: Use of 4 spaces per indentation level to ensure readability and uniform appearance across different IDEs.
- Braces: Use of "Egyptian Braces" style

2.3.2 Architectural Standards

The project will strictly follow the 3-Package Architecture model to ensure the safe development of each separated concern:

1. Presentation Layer (UI): Contains JavaFX controllers and FXML files.
2. Application Layer (Business Logic): Contains the functional base for the SCMS. Acts as the "brain" of the application.
3. Data Layer (Database): Contains Data Access Object (DAO) classes and SQL connection handling. Interacts only with the application layer.

Key Benefits:

- **Scalability:** Each layer can be scaled independently.
- **Maintainability:** The independence among the layers allows changes to be made with minimal impact on other layers.
- **Security:** The data layer is protected and hidden behind the application layer, therefore mitigating the risk of unauthorized access and SQL injection.
- **Flexibility:** Team members can work on different layers separately, speeding up the development of the project.

2.3.3 Documentation Standards

- **API Documentation (Javadoc):** Every public class and non-trivial method must include Javadoc comments explaining the parameters (`@param`), return values (`@return`), and potential exceptions (`@throws`).
- **Self-Documenting Code:** Variables must have descriptive names. Generic names such as `list1` or `temp` are avoided; instead, names like `memberList` or `temporaryEventBuffer` are used.
- **Inline Commentary:** `/**` is used for brief explanations of relatively complex logic within the script, but over-explaining obvious logic is avoided.

3 Responsibilities

3.1 Activities and responsibilities

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 8 / 11

Activity	Responsibility	Comment
Project Management & SDP	All members	Responsible for the initial Software Development Plan and timeline tracking.
Configuration & Git Management	Genidy	Organizing the GitHub repository and handling the version control workflow.
Requirements Specification (SRS)	Bora, Emir	Documenting functional and non-functional requirements.
System Design (SDD)	Hamza, Genidy	Creating the 3-package architecture and UML class diagrams.
Database Design & Integration	Bora, Emir	Designing the database schema and handling SQL connectivity.
Testing & Reporting (STP/STR)	Hamza, Genidy	Writing JUnit test cases and the final Software Test Report.
Implementation (Coding)	All members	Mandatory: All 4 members will strictly participate in the coding phase.
Software Analysis (SAR)	All members	Final analysis report and post-project analysis.

4 Risk Assessment

4.1 Risk Analysis

Risk Item	Probability	Impact	Mitigation Strategy
Data Security and Privacy	Low	High	Use prepared statements in Java to prevent SQL injection vulnerabilities.
Integration Conflicts	Medium	Medium	Set up clear GitHub branch rules so all 4 members can contribute without messing up each other's code.
Team Coordination & Availability	Medium	High	Keep a clear task board where every task has one main person responsible plus a backup reviewer, so nothing gets stuck if someone drops off.
Logic Complexity & Modular Coupling	Low	Medium	Maximize information hiding in the design so that changes in one package do not heavily impact others.

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 10 / 11

Software Design Tool Failure	Medium	Medium	In case Argo UML is incompatible or does not perform well with other parts of the project, it will be replaced with a modern alternative like Visual Paradigm or Draw.io.
-------------------------------------	--------	--------	---

4.2 Risk Planning

Risk Item	Potential Indicators	Strategy (Avoidance / Minimization)
Data Security and Privacy	Errors in database logs or unexpected results during SQL query execution.	Proper input validation will be performed on the frontend, and parameterised queries will be used for all database calls.
Integration Conflicts	Frequent merge conflicts on GitHub or builds breaking in the main branch.	Make it a rule that at least one team member reviews the code before any new feature is merged.
Team Coordination & Availability	Uneven contribution logs in GitHub or a lack of participation in mandatory weekly meetings.	Assign specific tasks and ensure a backup reviewer can take over if the lead member is unavailable.

Software Development Plan of SCMS software		
Doc # SCMS-SDP	Version: 1.5	Page 11 / 11

Logic Complexity & Modular Coupling	Hard time writing unit tests for one class without firing up the whole system, or too many connections per class showing up in early designs.	Every two weeks, we'll do "code cleanup" sessions to try simplify messy methods.
Software Design Tool Failure	Installation errors, crashes while drawing diagrams, or not being able to export UML files in the formats we need.	Argo UML will be tested in week one, in case it does not work, all design tasks will be implemented on to Visual Paradigm Community Edition, since it's regularly updated.